

Editorial: Problem A - Touko's Perfection

Setter: Hudson

Problem Overview

We are given an array A . We want to construct a non-decreasing array B such that $\sum |A_i - B_i|$ is minimized.

Difficulty Analysis

Rating: **2100**. A naive DP approach would define $dp[i][val]$ as the min cost to set the i -th element to val . This is $O(N \cdot \max(A))$, which is too slow. This is a standard application of the **Slope Trick**.

Solution Approach

Let $F_i(x)$ be the minimum cost to fix the prefix $1 \dots i$ such that the i -th element is exactly x .

$$F_i(x) = |x - A_i| + \min_{y \leq x} F_{i-1}(y)$$

The function $F_i(x)$ is always convex. We can maintain this function using a priority queue that stores the locations where the slope of the function changes. For the condition $B_i \geq B_{i-1}$, the transition $\min_{y \leq x} F_{i-1}(y)$ corresponds to keeping the "left" part of the convex function and flattening the "right" part (since we can always choose a larger value for free). Specifically, if we maintain the change points of the slope in a Max-Priority Queue: 1. Push A_i onto the queue twice. 2. If the maximum element in the queue (let's say top) is greater than A_i , it means the optimal "valley" of the function is to the left of the current top . 3. We pop the top element. The difference $top - A_i$ adds to our minimum cost.

Implementation (C++)

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main() {
9     ios_base::sync_with_stdio(false);
10    cin.tie(NULL);
11
12    int n;
```

```

13  if (!(cin >> n)) return 0;
14
15  // Max Priority Queue stores the slope change points
16  // Specifically, for the "left" side of the convex function
17  priority_queue<long long> pq;
18  long long ans = 0;
19
20  for (int i = 0; i < n; ++i) {
21      long long a;
22      cin >> a;
23
24      // Add the function |x - a|
25      // This adds a slope change of +1 at x=a and -1 at x=a
26      // In the context of the running min-prefix DP, we push 'a' twice.
27      pq.push(a);
28      pq.push(a);
29
30      // The transition min_{y <= x} F(y) means we effectively
31      // flatten the slope for the right side.
32      // We remove the rightmost slope change point if it's too high.
33      long long max_el = pq.top();
34      pq.pop();
35
36      // If the current optimal point (max_el) is greater than a,
37      // it contributes to the cost.
38      ans += (max_el - a);
39  }
40
41  cout << ans << endl;
42
43  return 0;
44 }

```